

Part A: Dockerize Each Component and Use Docker Compose

Step 1: Create Project Folder

Open Command Prompt or PowerShell:

```
cd %USERPROFILE%\Desktop
```

```
mkdir three-tier-docker
```

```
cd three-tier-docker
```

Create the folders:

```
mkdir backend
```

```
mkdir frontend
```

```
mkdir database
```

The structure will be:

```
three-tier-docker/
```

```
|
```

```
|— backend/
```

```
|— frontend/
```

```
|— database/
```

```
└─ docker-compose.yml
```

Step 2: Create Flask Backend

Go into the backend folder:

```
cd backend
```

Create:

```
app.py
```

Add:

```
from flask import Flask, jsonify
```

```
import psycpg2
```

```
import os
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Flask Backend is Running"
```

```
@app.route("/students")
```

```
def students():
```

```
    try:
```

```
        conn = psycopg2.connect(
```

```
            host=os.getenv("DB_HOST"),
```

```
            database=os.getenv("DB_NAME"),
```

```
            user=os.getenv("DB_USER"),
```

```
            password=os.getenv("DB_PASSWORD")
```

```
        )
```

```
        cur = conn.cursor()
```

```
        cur.execute("SELECT id, name, email, course FROM students ORDER BY id")
```

```
        rows = cur.fetchall()
```

```
        cur.close()
```

```
        conn.close()
```

```
result = []

for row in rows:
    result.append({
        "id": row[0],
        "name": row[1],
        "email": row[2],
        "course": row[3]
    })

return jsonify(result)

except Exception as e:
    return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create Backend requirements.txt

Inside the backend folder, create:

requirements.txt

Add:

Flask

psycpg2-binary

flask-cors

Step 4: Create Backend Dockerfile

Inside backend, create a file named:

Dockerfile

Add:

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

The backend is now ready to be containerized.

Step 5: Create React Frontend

Go back to the main project folder:

cd ..

Go into frontend:

cd frontend

Create a React application:

```
npm create vite@latest . -- --template react
```

When asked to continue, type:

y

Then install the packages:

```
npm install
```

Install Axios:

```
npm install axios
```

Step 6: Modify React App

Open:

```
frontend/src/App.jsx
```

Replace the existing code with:

```
import { useEffect, useState } from "react";
```

```
import axios from "axios";
```

```
function App() {
```

```
  const [students, setStudents] = useState([]);
```

```
  useEffect(() => {
```

```
    axios
```

```
      .get("http://localhost:5000/students")
```

```
      .then((response) => {
```

```
        setStudents(response.data);
```

```
})
```

```
.catch((error) => {
```

```
  console.error(error);
```

```
});
```

```
, []);
```

```
return (
```

```
<div>
```

```
<h1>Student Management System</h1>
```

```
<table border="1" cellPadding="10">
```

```
<thead>
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Name</th>
```

```
<th>Email</th>
```

```
<th>Course</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{students.map((student) => (
```

```
<tr key={student.id}>

  <td>{student.id}</td>

  <td>{student.name}</td>

  <td>{student.email}</td>

  <td>{student.course}</td>

</tr>

)}}

</tbody>

</table>

</div>

);

}
```

```
export default App;
```

Step 7: Create Frontend Dockerfile

Inside the frontend folder, create:

Dockerfile

Add:

FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]

Step 8: Configure PostgreSQL

We will use PostgreSQL as a Docker container.

Create a folder:

database

Inside it, create:

init.sql

Add:

CREATE TABLE IF NOT EXISTS students (

id SERIAL PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL,

course VARCHAR(100) NOT NULL

);

```
INSERT INTO students (name, email, course)
```

```
VALUES
```

```
('Rahul', 'rahul@gmail.com', 'BCA'),
```

```
('Priya', 'priya@gmail.com', 'BSc CS'),
```

```
('Amit', 'amit@gmail.com', 'BCA');
```

This SQL file will automatically create the table when the PostgreSQL container is initialized.

Step 9: Create docker-compose.yml

Go back to the main project folder:

```
cd ..
```

Create:

```
docker-compose.yml
```

Add:

```
services:
```

```
  frontend:
```

```
    build: ./frontend
```

```
    container_name: student_frontend
```

```
    ports:
```

```
      - "5173:5173"
```

```
    depends_on:
```

```
      - backend
```

```
  backend:
```

```
    build: ./backend
```

container_name: student_backend

ports:

- "5000:5000"

environment:

DB_HOST: database

DB_NAME: studentdb

DB_USER: postgres

DB_PASSWORD: postgres

depends_on:

- database

database:

image: postgres:16

container_name: student_database

restart: always

environment:

POSTGRES_DB: studentdb

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:

postgres_data:

Important

Inside Docker Compose, the Flask application does **not** use:

localhost

to connect to PostgreSQL.

It uses:

database

because database is the PostgreSQL service name.

Step 10: Final Project Structure

Your project should now look like:

three-tier-docker/

|

|— backend/

| |— app.py

| |— requirements.txt

| |— Dockerfile

|

|— frontend/

| |— src/

| | |— App.jsx

| |— package.json

| |— package-lock.json

| |— Dockerfile

|

|— database/

```
| └─ init.sql
```

```
|
```

```
| └─ docker-compose.yml
```

Step 11: Build All Containers

Open the terminal in:

three-tier-docker

Run:

docker compose build

Docker will build:

- React frontend image
 - Flask backend image
 - PostgreSQL will be downloaded from Docker Hub
-

Step 12: Start the Application

Run:

docker compose up -d

The -d option runs the containers in the background.

You should see containers being created.

Step 13: Check Running Containers

Run:

docker ps

You should see three containers similar to:

student_frontend

student_backend

student_database

All three should have status:

Up

Step 14: Check Docker Compose Services

Run:

```
docker compose ps
```

Expected:

NAME	STATUS
student_frontend	Up
student_backend	Up
student_database	Up

Step 15: Test the Backend

Open a browser and visit:

<http://localhost:5000/>

Expected output:

Flask Backend is Running

Now visit:

<http://localhost:5000/students>

Expected output:

```
[  
  {  
    "id": 1,  
    "name": "Rahul",  
    "email": "rahul@gmail.com",
```

```
"course": "BCA"
},
{
  "id": 2,
  "name": "Priya",
  "email": "priya@gmail.com",
  "course": "BSc CS"
},
{
  "id": 3,
  "name": "Amit",
  "email": "amit@gmail.com",
  "course": "BCA"
}
]
```

This proves that:

Flask → PostgreSQL

communication is working.

Step 16: Test the Frontend

Open:

<http://localhost:5173>

You should see:

Student Management System

ID	Name	Email	Course
----	------	-------	--------

-
- 1 Rahul rahul@gmail.com BCA
 - 2 Priya priya@gmail.com BSc CS
 - 3 Amit amit@gmail.com BCA

The complete communication is:

React

|

| HTTP Request



Flask

|

| SQL



PostgreSQL

|

| Student Data



Flask

|



React

Part B: Deploy on a Virtual Machine or Local Setup

For the practical, you can demonstrate the application on your **local computer** or an **Ubuntu virtual machine**.

Option 1: Local Setup

Docker Desktop must be running.

Run:

```
docker compose up -d
```

Then open:

```
http://localhost:5173
```

If the Student Management System appears with student records, the deployment is successful.

Option 2: Ubuntu Virtual Machine

Copy the project to the Ubuntu VM.

Install Docker if required.

Then go to the project folder:

```
cd three-tier-docker
```

Build:

```
docker compose build
```

Start:

```
docker compose up -d
```

Check:

```
docker ps
```

Find the VM's IP address:

```
hostname -I
```

For example:

```
192.168.1.20
```

If the VM is configured for network access, open from another computer:

```
http://192.168.1.20:5173
```

Step 17: View Container Logs

To check the frontend:

```
docker compose logs frontend
```

Backend:

```
docker compose logs backend
```

Database:

```
docker compose logs database
```

For the PostgreSQL container, you should eventually see a message indicating that the database is ready to accept connections.

Step 18: Stop the Application

When testing is complete:

```
docker compose down
```

This stops and removes the containers and network.

The PostgreSQL volume remains, so database data can persist.

Step 19: Restart the Application

To start it again:

```
docker compose up -d
```

Check:

```
docker compose ps
```

Then open:

```
http://localhost:5173
```

Docker Commands Summary

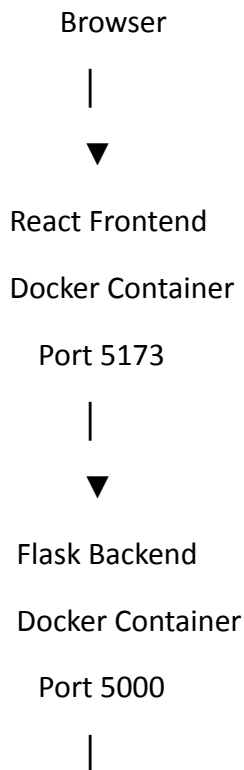
Command	Purpose
docker compose build	Build application images

Command	Purpose
<code>docker compose up -d</code>	Start all services
<code>docker compose ps</code>	Display service status
<code>docker ps</code>	Display running containers
<code>docker compose logs frontend</code>	View frontend logs
<code>docker compose logs backend</code>	View backend logs
<code>docker compose logs database</code>	View database logs
<code>docker compose down</code>	Stop and remove containers
<code>docker compose up -d</code>	Restart the application

Expected Result

The 3-Tier application is successfully deployed using Docker Compose.

The final setup contains:





PostgreSQL Database

Docker Container

Port 5432

Result

The frontend, backend, and database components were successfully Dockerized and orchestrated using Docker Compose. The 3-Tier application was deployed on the local/virtual machine environment, and communication between React, Flask, and PostgreSQL was successfully tested.